JQuery _Day - 11 Hands On _Sana Naveen

Problem 1

Problem Statement :

Student Feedback Form Interaction

Scenario:

A training institute wants to collect feedback from students through a simple web form. They want to enhance user experience using jQuery to validate inputs and display messages dynamically without reloading the page.

Requirements

• Create a feedback form with fields: Name, Email, Course Rating (dropdown), and Comments.
• Use jQuery selectors to capture form inputs.
• On clicking the Submit button, validate that Name and Email are not empty.
• Display a success message dynamically below the form using jQuery DOM manipulation.
• Clear the form fields after successful submission.

Technical Constraints

• Use jQuery CDN for integration.
• Use basic jQuery syntax: $(selector).action().
• Do not use plain JavaScript for validation (only jQuery methods).
• No backend integration required.

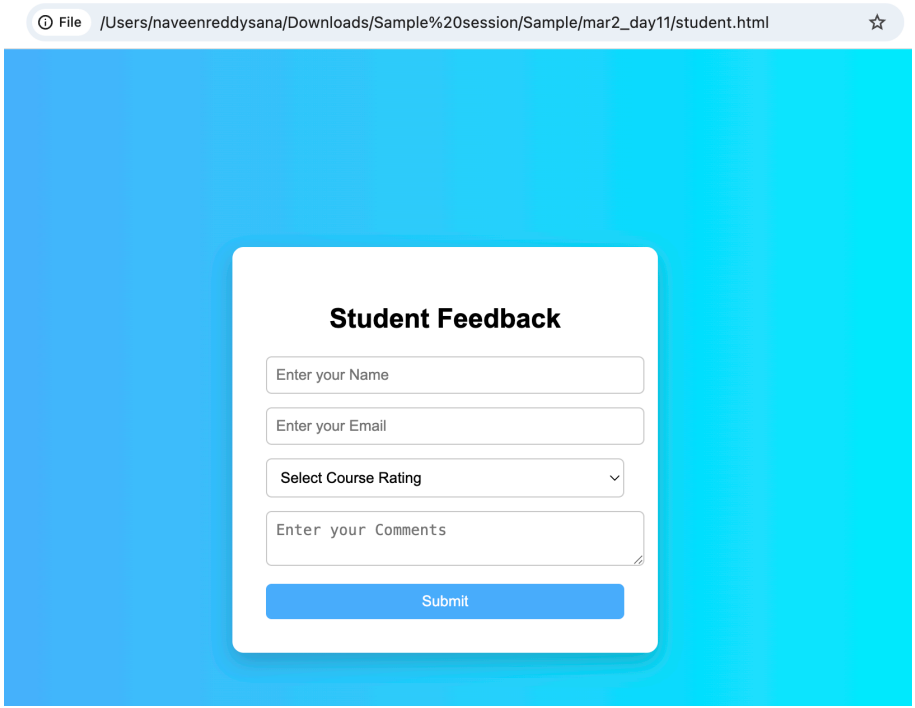
Code :

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Feedback Form</title>

    <!-- jQuery CDN -->
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

    <style>
        body {
            margin: 0;
            font-family: Arial, sans-serif;
            background: linear-gradient(to right, #4facfe, #00f2fe);
            height: 100vh;
            display: flex;
            justify-content: center;
            align-items: center;
        }

        .container {
            background-color: white;
            padding: 30px;
            border-radius: 10px;
            box-shadow: 0 8px 20px rgba(0,0,0,0.2);
            width: 320px;
        }

        h2 {
            text-align: center;
        }
```

```
        input, select, textarea {
            width: 100%;
            padding: 8px;
            margin-bottom: 12px;
            border: 1px solid #ccc;
            border-radius: 5px;
        }

        button {
            width: 100%;
            padding: 8px;
            background-color: #4facfe;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }

        button:hover {
            background-color: #00c6ff;
        }

        .error {
            color: red;
            margin-top: 10px;
        }

        .success {
            color: green;
            margin-top: 10px;
            font-weight: bold;
        }
    </style>
</head>

<body>

<div class="container">

    <h2>Student Feedback</h2>

    <form id="feedbackForm">

        <input type="text" id="name" placeholder="Enter your Name">

        <input type="email" id="email" placeholder="Enter your Email">

        <select id="rating">
            <option value="">Select Course Rating</option>
            <option>Excellent</option>
            <option>Good</option>
            <option>Average</option>
            <option>Poor</option>
        </select>

        <textarea id="comments" placeholder="Enter your Comments"></textarea>

        <button type="submit">Submit</button>

    </form>
```

```
    <div id="message"></div>

</div>

<script>
$(document).ready(function(){

    $("#feedbackForm").submit(function(e){

        e.preventDefault();

        let name = $("#name").val().trim();
        let email = $("#email").val().trim();

        $("#message").html("");

        if(name === "" || email === ""){
            $("#message")
                .removeClass("success")
                .addClass("error")
                .html("⚠ Please enter both Name and Email.");
            return;
        }

        $("#message")
            .removeClass("error")
            .addClass("success")
            .html("✅ Thank you for your feedback!");

        $("#feedbackForm")[0].reset();

    });

});
</script>

</body>
</html>
```

Output Screenshot :



Explanation :

This Student Feedback Form is built using jQuery for client-side validation and dynamic DOM manipulation. The form is centered using CSS Flexbox (display: flex; justify-content: center; align-items: center;) and styled with a gradient background for better UI experience. jQuery is integrated via CDN, and form submission is handled using $("#feedbackForm").submit() with e.preventDefault() to stop page reload. Input values are captured using .val(), validated to ensure Name and Email are not empty, and user-friendly messages are displayed dynamically using .html() along with .addClass() and .removeClass() for styling. After successful validation, the form fields are cleared using .reset(). This demonstrates proper event handling, selectors, and DOM manipulation using basic jQuery syntax.

Problem 2

Problem Statement :
A website contains a Frequently Asked Questions (FAQ) section. The company wants answers to appear and disappear when users click on the questions.
Requirements
• Create at least 5 FAQ questions with hidden answers.
• Use jQuery click() event to toggle answers.
• Apply slideToggle() or show()/hide() methods.
• Highlight the selected question using CSS class manipulation.

Technical Constraints
• Use only jQuery methods for DOM manipulation.
• Use class selectors and ID selectors appropriately.
• Avoid inline JavaScript.

Expectations:
• Smooth UI interaction.
• Proper use of jQuery effects.
• Organized HTML structure.

Learning Outcome
• Implement event handling using jQuery.
• Use basic jQuery effects.
• Practice DOM traversal and manipulation.

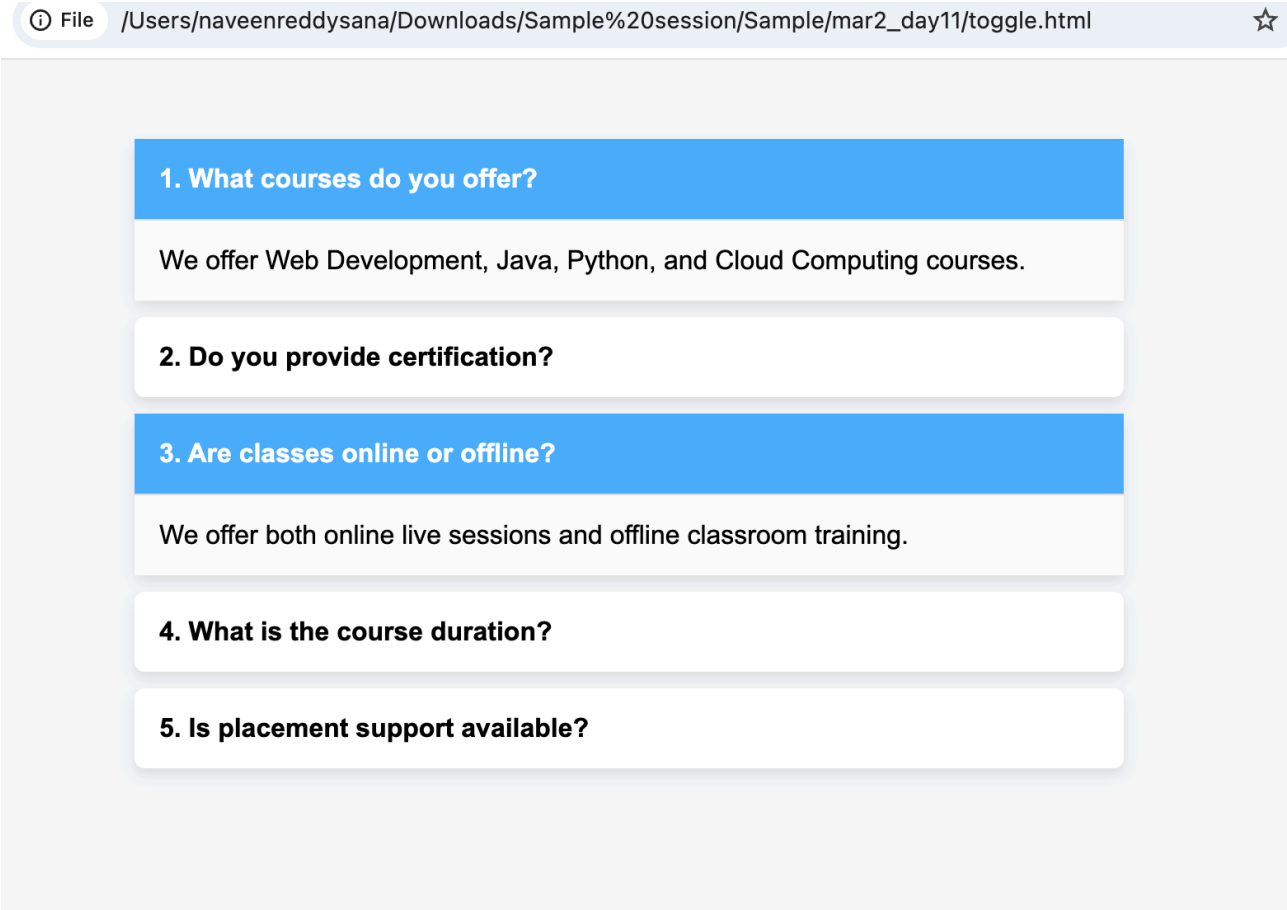
Code :

```
<!DOCTYPE html>
<html>
<head>
   <title>FAQ Toggle Section</title>

   <!-- jQuery CDN -->
   <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

   <style>
      body {
         font-family: Arial, sans-serif;
         background-color: #f4f6f9;
         padding: 40px;
      }

      .faq-container {
         max-width: 600px;
         margin: auto;
      }

      .faq-item {
         background: white;
         margin-bottom: 10px;
         border-radius: 6px;
         box-shadow: 0 4px 8px rgba(0,0,0,0.1);
```

```
        }

        .question {
            padding: 15px;
            cursor: pointer;
            font-weight: bold;
        }

        .question.active {
            background-color: #4facfe;
            color: white;
        }

        .answer {
            display: none;
            padding: 15px;
            border-top: 1px solid #ddd;
            background-color: #fafafa;
        }
    </style>
</head>

<body>

<div class="faq-container">

    <div class="faq-item">
        <div class="question">1. What courses do you offer?</div>
        <div class="answer">We offer Web Development, Java, Python, and Cloud Computing courses.</div>
    </div>

    <div class="faq-item">
        <div class="question">2. Do you provide certification?</div>
        <div class="answer">Yes, we provide course completion certificates.</div>
    </div>

    <div class="faq-item">
        <div class="question">3. Are classes online or offline?</div>
        <div class="answer">We offer both online live sessions and offline classroom training.</div>
    </div>

    <div class="faq-item">
        <div class="question">4. What is the course duration?</div>
        <div class="answer">Course duration varies from 2 to 6 months depending on the program.</div>
    </div>

    <div class="faq-item">
        <div class="question">5. Is placement support available?</div>
        <div class="answer">Yes, we provide interview preparation and placement assistance.</div>
    </div>

</div>

<script>
$(document).ready(function(){

    $(".question").click(function(){

        // Toggle answer smoothly
        $(this).next(".answer").slideToggle();
```

```
        // Toggle highlight class
        $(this).toggleClass("active");

    });

});
</script>

</body>
</html>
```

Output Screenshot :



Explanation :

The Dynamic FAQ Toggle section is implemented using jQuery event handling and DOM traversal. All answers are initially hidden using CSS (display: none). A click event is attached to elements with the .question class using $(".question").click(). When a question is clicked, $(this).next(".answer").slideToggle() is used to smoothly show or hide the corresponding answer, ensuring dynamic interaction without page reload. The toggleClass("active") method is used to highlight the selected question by adding or removing a CSS class. This implementation demonstrates proper use of jQuery selectors, effects, and DOM manipulation for creating interactive UI components.

Problem 3

Problem Statement :

Problem 3 (Level-2): Product List with Add to Cart Counter
Scenario:
An e-commerce website wants to display a list of products. When users click 'Add to Cart', the cart counter at the top should increase dynamically.
Requirements
• Display at least 4 products with 'Add to Cart' buttons.
• Use jQuery to handle button click events.
• Increase the cart counter dynamically without page refresh.
• Disable the button once the product is added.
• Display a small confirmation message beside the product.

Technical Constraints
• Use jQuery event delegation if needed.
• Use .text(), .attr(), and .prop() methods.
• No backend or database integration.

Expectations:
• Efficient event handling.
• Proper use of jQuery methods for attribute manipulation.
• Clean UI updates.

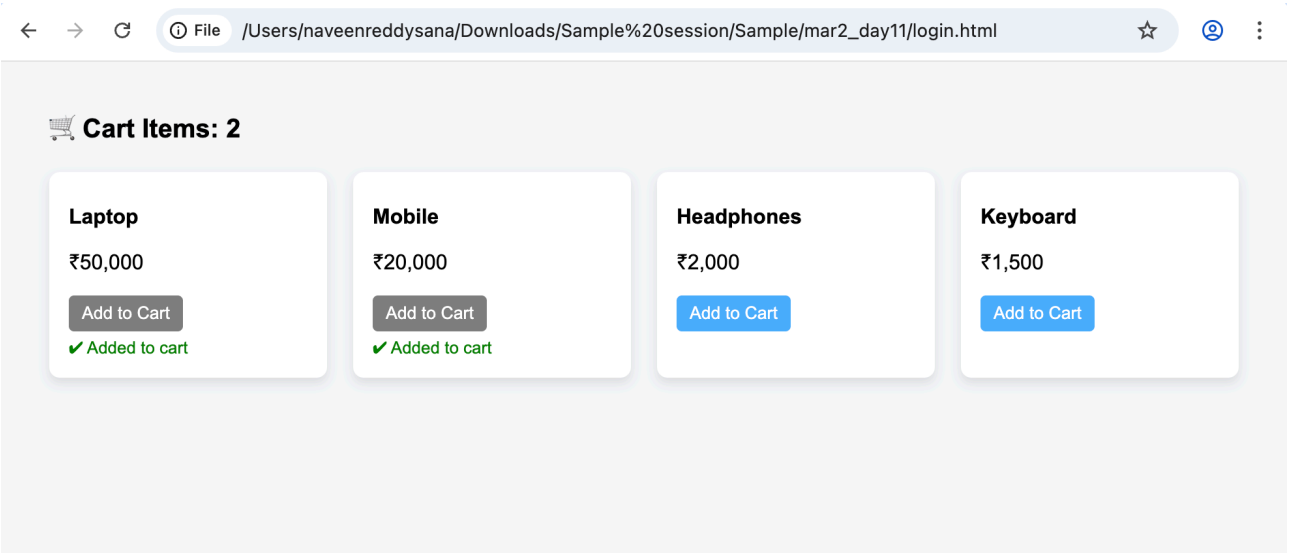
Code :

```
<!DOCTYPE html>
<html>
<head>
    <title>Product Cart Counter</title>

    <!-- jQuery CDN -->
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

    <style>
        body {
            font-family: Arial, sans-serif;
            background: #f4f6f9;
            padding: 30px;
        }

        .cart {
            font-size: 20px;
            margin-bottom: 20px;
            font-weight: bold;
        }

        .product-container {
            display: flex;
            gap: 20px;
        }

        .product {
            background: white;
            padding: 15px;
            width: 200px;
            border-radius: 8px;
            box-shadow: 0 4px 8px rgba(0,0,0,0.1);
        }
```

```
        .product h4 {
            margin: 10px 0;
        }

        button {
            padding: 6px 10px;
            border: none;
            background-color: #4facfe;
            color: white;
            border-radius: 4px;
            cursor: pointer;
        }

        button:disabled {
            background-color: gray;
            cursor: not-allowed;
        }

        .msg {
            color: green;
            font-size: 13px;
            margin-top: 5px;
        }
    </style>
</head>

<body>

<div class="cart">
    🛒 Cart Items: <span id="cartCount">0</span>
</div>

<div class="product-container" id="productList">

    <div class="product">
        <h4>Laptop</h4>
        <p>₹50,000</p>
        <button class="addBtn">Add to Cart</button>
        <div class="msg"></div>
    </div>

    <div class="product">
        <h4>Mobile</h4>
        <p>₹20,000</p>
        <button class="addBtn">Add to Cart</button>
        <div class="msg"></div>
    </div>

    <div class="product">
        <h4>Headphones</h4>
        <p>₹2,000</p>
        <button class="addBtn">Add to Cart</button>
        <div class="msg"></div>
    </div>

    <div class="product">
        <h4>Keyboard</h4>
        <p>₹1,500</p>
        <button class="addBtn">Add to Cart</button>
```

```
        <div class="msg"></div>
    </div>

</div>

<script>
$(document).ready(function(){

    let count = 0;

    // Event Delegation
    $("#productList").on("click", ".addBtn", function(){

        count++;

        // Update cart counter using .text()
        $("#cartCount").text(count);

        // Disable button using .prop()
        $(this).prop("disabled", true);

        // Add attribute example
        $(this).attr("title", "Product already added");

        // Show confirmation message
        $(this).siblings(".msg")
              .text("✔ Added to cart");

    });

});
</script>

</body>
</html>
```

Output Screenshot :



Explanation :

The Product List with Add to Cart Counter is implemented using jQuery event handling and DOM manipulation. A click event is attached using event delegation with $("#productList").on("click", ".addBtn", function(){}) to efficiently handle button interactions. When a user clicks "Add to Cart", the cart counter is dynamically updated using .text(), ensuring the page does not reload. The clicked button is disabled using .prop("disabled", true) to prevent duplicate additions, and a confirmation message is displayed beside the product using DOM traversal with .siblings().text(). The .attr() method is used to add attributes like tooltips. This demonstrates proper use of jQuery selectors, property manipulation, and dynamic UI updates in an e-commerce scenario.

Problem 4

Problem Statement :
Interactive To-Do List Application
Scenario:
A productivity web app requires a dynamic To-Do List where users can add, mark as complete, and remove tasks using jQuery.

Requirements
• Provide an input field and Add button to insert tasks.
• Display tasks dynamically in a list.
• Allow users to mark tasks as completed (strike-through effect).
• Provide a delete button for each task.
• Display total task count and completed task count dynamically.

Technical Constraints
• Use jQuery for all DOM manipulation and event handling.
• Use append(), remove(), toggleClass(), and on() methods.
• No external libraries except jQuery.

Expectations:
• Proper dynamic element handling.
• Efficient event binding for dynamically created elements.
• Responsive and interactive UI behavior.

Code :

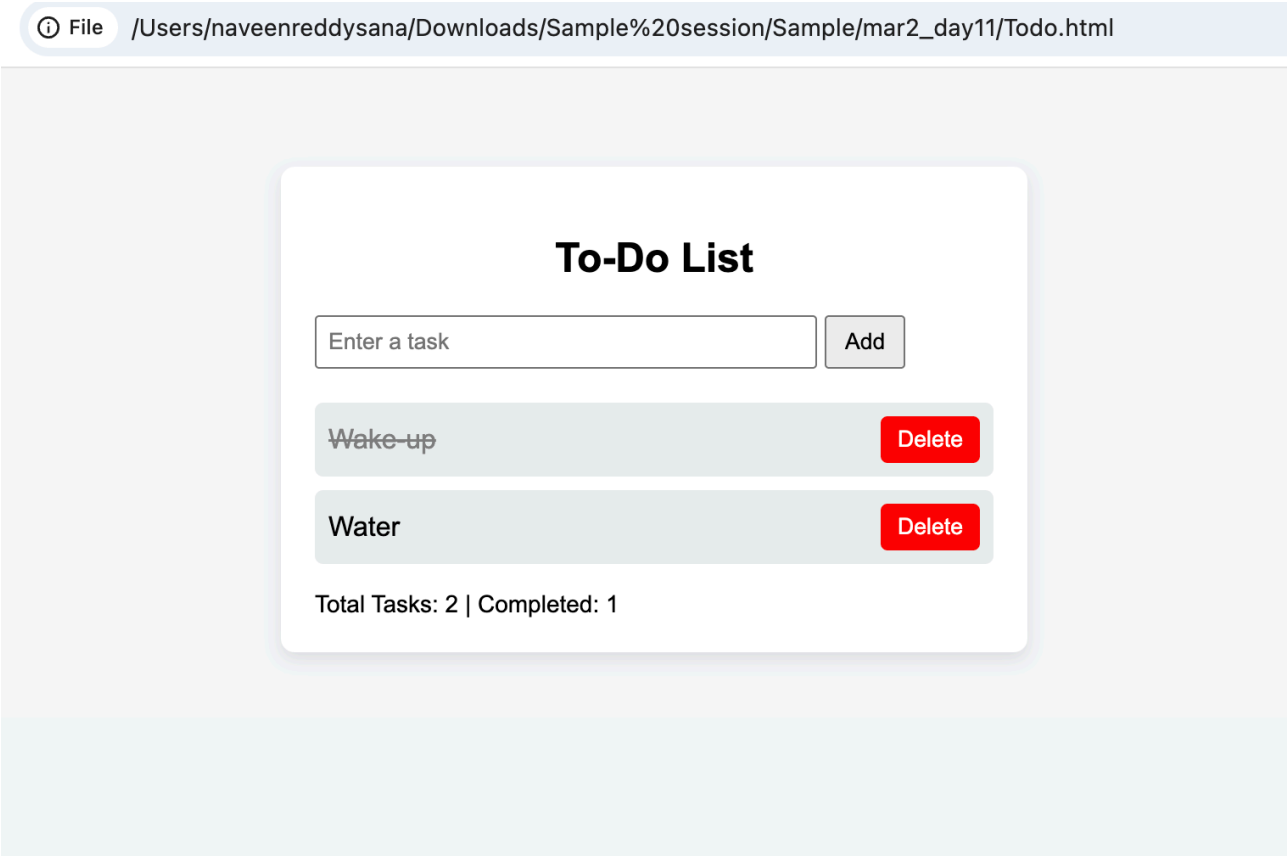
```
<!DOCTYPE html>
<html>
<head>
   <title>To-Do List App</title>

   <!-- jQuery CDN -->
   <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

   <style>
      body {
         font-family: Arial, sans-serif;
         background: #f4f6f9;
         display: flex;
         justify-content: center;
         padding-top: 50px;
      }

      .todo-container {
         background: white;
         width: 400px;
         padding: 20px;
         border-radius: 8px;
         box-shadow: 0 4px 10px rgba(0,0,0,0.1);
      }
```

```
        h2 {
            text-align: center;
        }

        input {
            width: 70%;
            padding: 6px;
        }

        button {
            padding: 6px 10px;
            cursor: pointer;
        }

        ul {
            list-style: none;
            padding: 0;
            margin-top: 20px;
        }

        li {
            padding: 8px;
            margin-bottom: 8px;
            background: #e9ecef;
            display: flex;
            justify-content: space-between;
            align-items: center;
            border-radius: 5px;
        }

        .completed {
            text-decoration: line-through;
            color: gray;
        }

        .deleteBtn {
            background: red;
            color: white;
            border: none;
            border-radius: 4px;
        }

        .stats {
            margin-top: 15px;
            font-size: 14px;
        }
    </style>
</head>

<body>

<div class="todo-container">

    <h2>To-Do List</h2>

    <input type="text" id="taskInput" placeholder="Enter a task">
    <button id="addTask">Add</button>

    <ul id="taskList"></ul>
```

```
    <div class="stats">
        Total Tasks: <span id="totalCount">0</span> |
        Completed: <span id="completedCount">0</span>
    </div>

</div>

<script>
$(document).ready(function(){

    let total = 0;
    let completed = 0;

    // Add Task
    $("#addTask").on("click", function(){

        let taskText = $("#taskInput").val().trim();

        if(taskText === "") return;

        // Create task dynamically
        $("#taskList").append(
            "<li>" +
                "<span class='taskText'>" + taskText + "</span>" +
                " <button class='deleteBtn'>Delete</button>" +
            "</li>"
        );

        total++;
        $("#totalCount").text(total);

        $("#taskInput").val("");
    });

    // Mark Complete (Event Delegation)
    $("#taskList").on("click", ".taskText", function(){

        $(this).toggleClass("completed");

        if($(this).hasClass("completed")){
            completed++;
        } else {
            completed--;
        }

        $("#completedCount").text(completed);
    });

    // Delete Task
    $("#taskList").on("click", ".deleteBtn", function(){

        let isCompleted = $(this).siblings(".taskText").hasClass("completed");

        if(isCompleted){
            completed--;
            $("#completedCount").text(completed);
        }

        total--;
        $("#totalCount").text(total);
```

```
        $(this).parent().remove();
    });

});
</script>

</body>
</html>
```

Output Screenshot :



Explanation :

The Interactive To-Do List application is built using jQuery for dynamic DOM manipulation and event handling. Tasks are added dynamically to the list using .append(), and event delegation is implemented with .on() to handle clicks on dynamically created elements. The toggleClass() method is used to apply or remove a strike-through effect for completed tasks, while .remove() deletes tasks from the DOM. The total and completed task counts are updated in real time using .text(), ensuring smooth UI updates without page refresh. This implementation demonstrates efficient dynamic element handling and proper use of jQuery methods to build an interactive client-side application.